NED University of Engineering and Technology, Karachi

Computer Science and Information Technology

COMPLEX COMPUTING PROBLEM (CCP) PROJECT SUBMISSION



Knowledge-Based Systems (CT-359)

Project Title	Intelligent Knowledge-Based System for Industrial Equipment Fault Diagnosis and Maintenance Decision Support
Implemented System	ARM / INSIGHT — Robotic Arm Fault Diagnosis and Maintenance Decision Support
Submission Date	31 October 2026
Live Demo	arm-insight-kbs.app
Source Code	GitHub Repo

Student Details

Sr. No.	Student Name	Roll No.	Signature
1	Saad Ali	CT-24221	
2	Muhammad Taha Farrukh	CT-24224	
3	Uzair Ahmed	CT-24244	
4	UsaidUllah Qureshi	CT-24227	

1. Project Instructions

ARM / INSIGHT is submitted as a browser-based KBS prototype for robotic-arm fault diagnosis. It does **not** command a physical robot. How each CCP instruction is met is summarized below; details appear in later sections.

Table 1: *Compliance with the CCP project instructions*

Instruction	How this submission complies
Source code, dataset / knowledge base files, report and presentation	Repository: `dist/` (application), `dataset/` (raw + cleaned UCI data, ETL notebook), `knowledge_base.json`, `sample_cases.json`, automated tests, this report and `docs/*.pptx`.
Implementation must be executable and demonstrable	Runs in any browser — locally via `serve.py` or at the live Vercel URL. A one-click Viva demo button scripts the walkthrough (Appendix A).
Assumptions	Section 4.5 lists six assumptions with justification. The prototype does not control a robot.
External datasets, sources, libraries and tools	Sections 6 and 9 cite the UCI dataset, Universal Robots documentation, Supabase and Vercel.
Report and presentation	`docs/KBS_Robotic_Arm_CCP_Presentation.pptx`.

Project Abstract

The system implements an explicit **ETL (Extract–Transform–Load) cleaning pipeline** on the authentic **UCI Robot Execution Failures** dataset (DOI 10.24432/C5M89N), rule-based **fault prediction** by forward chaining, synthetic evaluation scenarios for regression, optional cloud persistence (Supabase) and public deployment (Vercel).

The robot type, plant-specific alarm mappings, real costs and historical failure data were not supplied in the brief; non-manufacturer rules are therefore explicit educational assumptions. **No operational accuracy or safety certification is claimed.**

The Information Flow

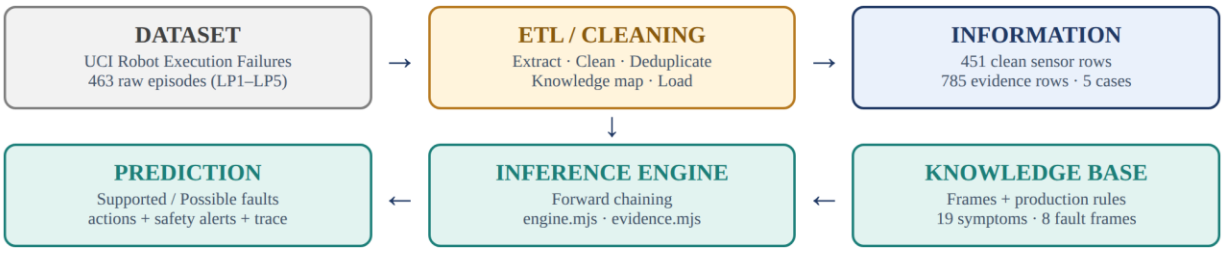


Figure 1: *Information flow: Dataset → ETL → Information → Knowledge Base → Inference → Prediction.*

2. Problem Statement

Table 3: *Course learning outcomes (CLOs) and where they are addressed*

CLO	Requirement	Addressed in
CLO 1	Explain how a KBS can represent industrial equipment knowledge — equipment, components, symptoms, faults, causes and maintenance actions — using an appropriate knowledge-representation formalism.	Sections 5.1, 5.2, 6.3
CLO 2	Analyze the fault-diagnosis problem by identifying incomplete requirements, stakeholder needs, relationships, constraints and suitable knowledge-based approaches.	Sections 3, 4.1 – 4.5
CLO 3	Evaluate knowledge-representation methods and inference algorithms for diagnosing faults and recommending maintenance actions; justify the selected approach on accuracy, explainability, safety, cost and system requirements.	Sections 5.3 – 5.5, 8.1

3. Problem Description / Project Context

A manufacturing cell uses an industrial robotic arm with joints and transmission, tool (payload) configuration, controller thermal paths, position feedback, power supply, communication harness, joint brakes and an optional pneumatic gripper. Equipment knowledge is scattered across operator observations, controller and sensor alarms, maintenance records, technician inspections and manuals or expert statements. The same symptom (for example a protective stop) can arise from more than one cause, evidence is often incomplete, and different sources may disagree. Stakeholders pull in different directions: technicians need auditable hypotheses and safe next actions; supervisors want lower downtime and cost; safety officers require hazard escalation that cannot be skipped for production continuity.

The ARM / INSIGHT prototype is a rule-based KBS that records categorical observations (Yes / No / Unknown / Conflict), optionally with multi-source claims, and then applies eight explicit diagnostic production rules by forward chaining. Fully supported hypotheses receive a cause narrative and a qualified maintenance recommendation; hazards escalate independently of rule completion. A browser diagnostic studio, case archive, maintenance board, rule explorer and evaluation lab support demonstration and regression. Optional Supabase sync stores shared workspace data; the live academic demo is hosted on Vercel. The system cannot diagnose unmodelled failures and must not replace the model-specific service manual or qualified personnel.

Table 4: *ARM / INSIGHT at a glance*

Equipment scope	One generic educational industrial robotic arm (no live control)
Knowledge base	Version 2.1 — 19 symptoms, 8 fault frames, 3 cited manufacturer sources
Representation	Frames + production rules stored as inspectable JSON
Inference	Forward chaining; four evidence states; independent safety alerts
Dataset	UCI Robot Execution Failures — 463 episodes (LP1–LP5), cleaned to 451 sensor rows
Verification	13/13 evaluation scenarios PASS; 24 additional automated checks PASS (37 in total)
Deployment	Vercel static site (live) and localhost via `serve.py`; optional Supabase sync

4. Problem Analysis

4.1 Problem Understanding

The core difficulty is dispersed, incomplete and sometimes conflicting maintenance knowledge. A useful KBS must:

1. Represent equipment → component → symptom → fault → cause → action relationships explicitly.
2. Distinguish **unknown** (not checked) from **no** (verified absence).
3. Allow **multiple simultaneous** fault hypotheses.
4. Show **why** a rule fired or failed (explanation facility).
5. Respect **safety**, while still supporting cost and downtime preferences as secondary concerns.

The prototype narrows the broad "all industrial equipment" brief to one generic educational robotic arm, so the knowledge base stays inspectable and defensible in the viva.

4.2 Stakeholders and Their Requirements

Table 5: *Stakeholders and required outcomes*

Stakeholder	Required outcome
Operator	Record symptoms and source notes without having to decide the root cause.
Maintenance technician	See matched, missing and conflicting evidence, the possible cause and a safe next action.
Safety officer	Immediate hazard escalation regardless of whether a diagnostic rule is complete.
Production supervisor	Compare relative downtime and cost without overriding hazard priority.
Knowledge engineer / instructor	Inspect and edit rules, provenance and regression results.
Course examiner	Demonstrable, explainable KBS aligned to CLO 1–3 and the CCP rubrics.

4.3 Constraints and Conflicting Requirements

Table 6: *Conflicting requirements and how the prototype resolves them*

Conflict	How the prototype treats it
Safety vs rapid restart	Protective stops and hazard symptoms escalate independently; recommendations require qualified review.

Conflict	How the prototype treats it
Lower cost vs reliability	Cost and downtime are secondary sort keys after status and severity.
Need for a decision vs missing data	Unknown never counts as No; rules stay Possible / Unassessed until evidence exists.
Conflicting reports	Opposing Yes / No source claims → Conflict; that symptom cannot fully support a rule.
Local demo vs shared audi	Browser localStorage always works; Supabase optionally mirrors the workspace for a shared demo.

4.4 Root Cause / Requirement Identification

The brief leaves several requirements incomplete. Before any real plant deployment, the organisation would still need to identify:

- Exact robot model and end-effector configuration
- Alarm-code semantics and approved sensor thresholds
- Access control and technician sign-off rules
- Field maintenance history and source-reliability weights
- Real cost and downtime figures
- Shared audit and retention policy

These were not supplied. The project therefore avoids inventing numeric thresholds, marks synthetic cases clearly and documents its educational assumptions.

4.5 Assumptions

Table 7: *Assumptions made to complete the problem*

#	Assumption
A1	One generic robot configuration is used; the UI arm map is illustrative, not a certified model.
A2	Observations are categorical: Yes, No, Unknown, Conflict. Source claims are Yes or No only; opposing claims are not averaged.
A3	Each diagnostic rule is an AND conjunction of required symptoms. A full match gives a Supported hypothesis, not a legally confirmed root cause.
A4	Severity, relative cost and downtime labels are teaching attributes, not measured plant KPIs.
A5	Evidence source labels (operator, sensor, etc.) are user assertions, not authenticated telemetry.
A6	Cloud sync uses educational open Row-Level Security for the demo; it is not production hardening.

5. Knowledge Representation and KBS Design

5.1 Selected Knowledge Representation Formalism

The selected formalism is frames + production rules, stored as inspectable JSON. Frames capture the structure of each fault (component, cause, action, severity, and provenance); production rules capture the diagnostic logic (IF all required symptoms are present THEN support the fault).

Table 8: *Knowledge-representation constructs*

Construct	What it represents
Symptom dictionary	Observable evidence vocabulary (equipment observations).
Fault frame	Component, name, required conditions, cause, severity, cost, downtime, action and provenance.
Production rule	`IF` all conditions = Yes `THEN` support fault + recommend action.
Evidence / claims	Provenance: source type, present / absent, note and timestamp.

The conceptual chain that the knowledge base encodes is shown in Figure 2.

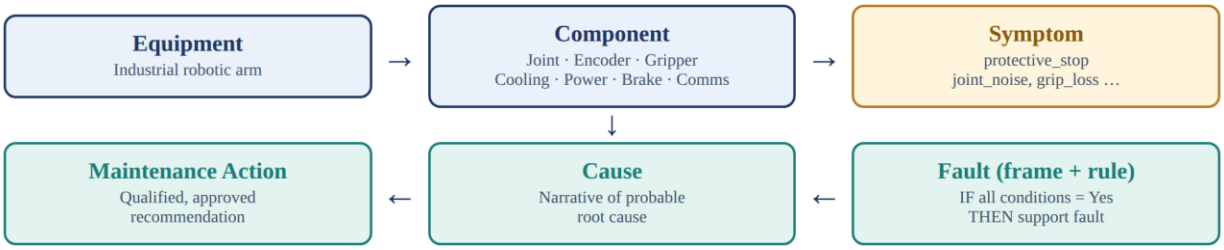


Figure 2: *Conceptual knowledge chain: equipment → component → symptom → fault → cause → maintenance action.*

5.2 Knowledge Base / Ontology / Rule Base

The file `knowledge_base.json` (version 2.1) contains 19 symptoms, 8 fault rules and 3 cited manufacturer concept sources. Table 9 lists the fault rules and Table 10 the symptom dictionary.

Table 9: *Fault rules in the knowledge base (all required observations must be Yes)*

Fault ID	Fault name	Component	Required observations	Severity	Cost / Down
`payload`	Payload configuration mismatch	Tool configuration	`protective_stop` ∧ `payload_mismatch`	High	Low / Short
`mechanical`	Joint mechanical degradation	Joint / transmission	`joint_noise` ∧ `joint_friction` ∧ `position_drift`	High	High / Long

Fault ID	Fault name	Component	Required observations	Severity	Cost / Down
thermal	Cooling-related overheating	Motor / controller cooling	overtemperature ∧ cooling_blocked	High	Medium / Medium
encoder	Position feedback fault	Encoder / feedback circuit	encoder_alarm ∧ feedback_unstable	High	Medium / Medium
gripper	Pneumatic grip fault	End effector	grip_loss ∧ pressure_low	High	Low / Short
communication	Communication connection fault	Communication harness	communication_alarm ∧ connection_damage	Medium	Medium / Medium
power	Power supply fault	Power system	power_alarm ∧ supply_unstable	High	Medium / Medium
brake	Joint holding / brake fault	Joint brake	brake_alarm ∧ joint_droop	Critical	High / Long

Table 10: *Symptom dictionary (19 observable evidence items)*

Symptom ID	Label shown in the UI	Used by fault rule
protective_stop	Protective stop reported	payload
payload_mismatch	Configured payload / centre of gravity differs from tool	payload
position_drift	Repeated positioning drift	mechanical
joint_noise	Abnormal joint noise	mechanical
joint_friction	Abnormal joint friction reported by qualified inspection	mechanical
overtemperature	Controller reports overtemperature	thermal
cooling_blocked	Cooling path obstructed	thermal
encoder_alarm	Encoder / position feedback alarm	encoder
feedback_unstable	Position feedback unstable	encoder
grip_loss	Gripper loses workpiece	gripper
pressure_low	Pneumatic pressure below equipment specification	gripper

Symptom ID	Label shown in the UI	Used by fault rule
communication_alarm	Communication fault alarm	communication
connection_damage	Visible cable / connector damage	communication
power_alarm	Supply voltage alarm	power
supply_unstable	Supply instability verified by qualified personnel	power
brake_alarm	Brake fault alarm	brake
joint_droop	Unexpected joint droop	brake
smoke	Smoke or burning smell	Safety hazard alert only
unexpected_motion	Unexpected motion observed	Safety hazard alert only

Example production rule (payload family)

```
IF   protective_stop = Yes
AND  payload_mismatch = Yes
THEN support "Payload configuration mismatch"
     recommend qualified review of approved payload settings
```

Payload and abnormal-friction concepts follow Universal Robots guidance [1]–[3]. The exact conjunctions for the other fault families, and all cost / downtime classes, are author-created educational rules, not manufacturer-approved mappings.

Excerpt: fault frame in knowledge_base.json

```
{ "id": "payload",
  "name": "Payload configuration mismatch",
  "component": "Tool configuration",
  "conditions": ["protective_stop", "payload_mismatch"],
  "severity": "High", "cost": "Low", "downtime": "Short",
  "cause": "Incorrect tool mass or centre-of-gravity
configuration.",
  "action": "Compare tool and workpiece data with approved payload
settings;
            have an authorized technician correct configuration.",
  "provenance": "UR protective-stop guidance; illustrative
diagnostic conjunction." }
```

5.3 Inference Mechanism

The selected algorithm is forward chaining over the rule base, implemented in `dist/engine.mjs` as the function `diagnose()`. Observations are the initial facts; each rule is evaluated against them and every satisfied rule adds a conclusion and an action to the explanation trace. Figure 3 shows the stages.

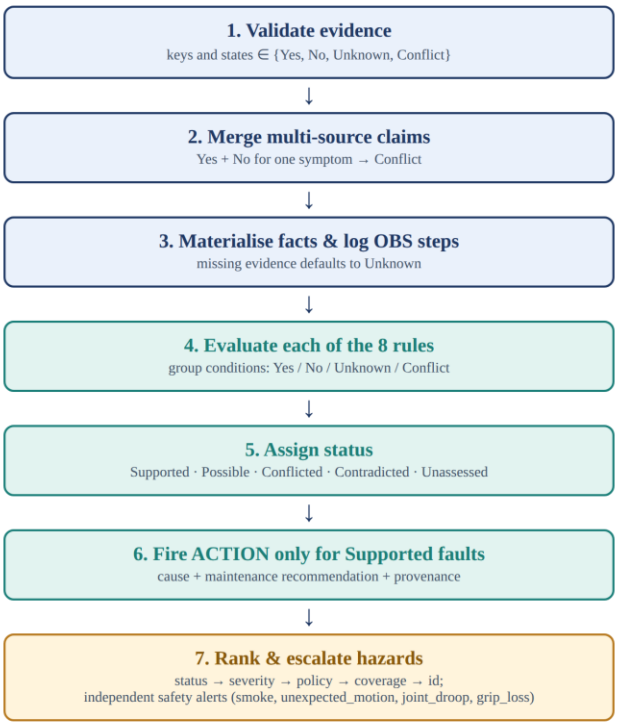


Figure 3: *Inference pipeline implemented by diagnose() in engine.mjs.*

For each fault rule the conditions are partitioned into Yes / No / Unknown / Conflict groups and one of five statuses is assigned (Table 11).

Table 11: *Hypothesis status assigned to each fault rule*

Status	Meaning
Supported	All required conditions are Yes — the maintenance action is fired.
Possible	Some conditions are Yes and none is No or Conflict — evidence is incomplete.
Conflicted	At least one condition is Conflict and none is No.
Contradicted	At least one condition is No — the rule cannot hold.
Unassessed	No condition is Yes yet.

Hypotheses are ranked by status → severity → selected policy (Safety first / Lower cost / Less downtime) → coverage → id. Independent safety alerts are emitted for the hazard symptoms `smoke`, `unexpected_motion`, `joint_droop` and `grip_loss` when they are Yes or Conflict. For F rules with at most K conditions the cost is O(FK) for evaluation plus O(F log F) for sorting; the rule graph is acyclic, so chaining always terminates.

Excerpt: status assignment in dist/engine.mjs

```
const status = groups.no.length       ? 'Contradicted'
             : groups.conflict.length ? 'Conflicted'
             : groups.yes.length === f.conditions.length ?
'Supported'
             : groups.yes.length      ? 'Possible' : 'Unassessed';
if (status === 'Supported') trace.push(`RULE ${f.id}:
${f.conditions.join(' AND ')} → fault:${f.id}`);
```

5.4 Conflict Resolution Strategy

1. **Multi-source merge** (`evidence.mjs`): if any claim is Yes and any claim is No for the same symptom, the state becomes **Conflict**.
2. Conflicted symptoms **cannot** count as Yes for rule support, so the hypothesis becomes Conflicted (or stays incomplete).
3. The UI lists conflicts; guided mode prioritises unresolved checks.
4. A manual override of a symptom clears prior claims for that key (an explicit user choice).
5. Hazards that are Yes or Conflict still escalate even if no rule is Supported.
6. Ranking never lets cost or downtime outrank Supported / Critical safety ordering.

Table 12: *Worked conflict-resolution example (payload_mismatch)*

Source claims	Merged state	Effect on the payload rule
Operator: Yes	Yes	Counts as present evidence.
Operator: Yes + Sensor: No	Conflict	Rule becomes Conflicted; not Supported until the source is verified.
No claims	Unknown	Rule stays Possible (if protective_stop = Yes).
Sensor: No only	No	Rule becomes Contradicted.

5.5 Explanation Facility

For each visible hypothesis the UI shows:

- Status badge and severity
- Coverage (% of conditions that are Yes) — **not** a probability
- Grouped evidence lines (yes / no / unknown / conflict)
- Cause text and, when Supported, the maintenance action
- Provenance string

A reasoning timeline lists OBS / RULE / ACTION steps (shown in Figure 14). Case export includes observations, findings, notes and the trace for viva and audit discussion.

5.6 System Architecture

The system is organised in five layers (Figure 4, Table 13). The knowledge and inference layers are independent of the user interface and are exercised directly by the Node.js regression tests.

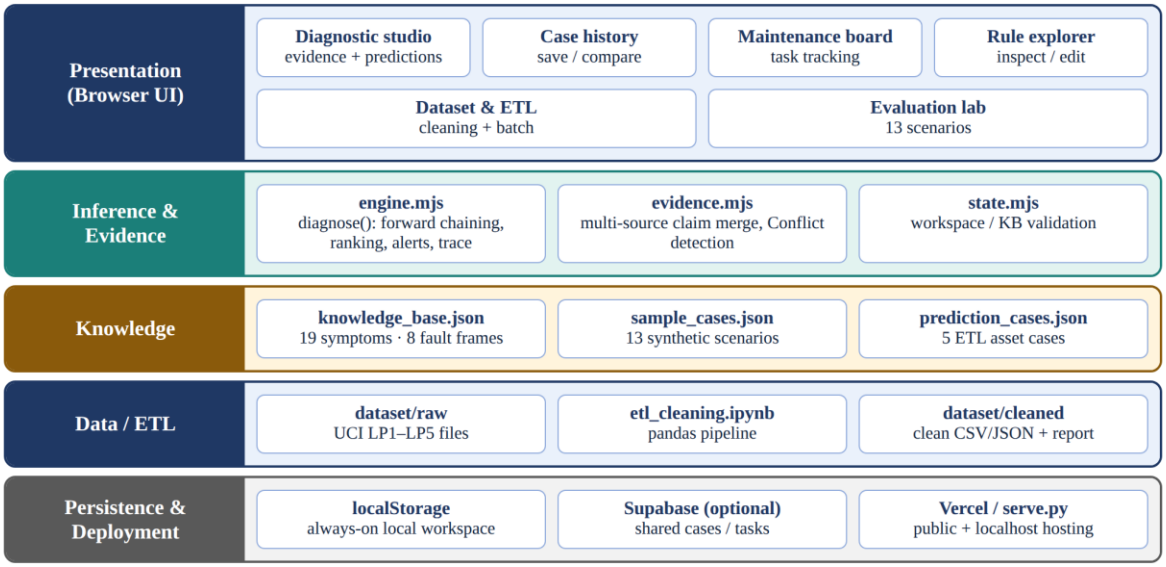


Figure 4: *Layered architecture of ARM / INSIGHT.*

Table 13: *Architecture layers and responsibilities*

Layer	Responsibilities
ETL	UCI extract → clean → knowledge map → load, plus UI report
Presentation	Studio, ETL page, cases, maintenance board, rules and evaluation
Inference / Prediction	Four-state rule evaluation, hazards, next-best questions, trace
Evidence	Source categories and conflict merge
Knowledge	JSON frames / rules plus 13 synthetic scenarios
Persistence	localStorage plus optional Supabase upsert / mirror
Deployment	Vercel static site; schema in `supabase/schema.sql`

5.7 Prediction Facility

In this KBS, prediction means inferring the most plausible fault hypotheses from cleaned or observed evidence using the rule base (not a trained ML classifier).

- **Supported** predictions fire maintenance recommendations.
- **Possible** predictions flag incomplete evidence.
- Batch prediction runs on the ETL-built asset cases (`prediction_cases.json`) from the **Dataset & ETL** page.
- Coverage % is rule-match completeness, **not** probability.

6. Dataset / Knowledge Sources

Table 14 lists every dataset and knowledge source. Items marked submit are part of the submission package.

Table 14: *Datasets and knowledge sources*

Source / Dataset	Type	Size / Records	Purpose	Citation / URL
External sources				
UCI Robot Execution Failures	Authentic public dataset (LP1–LP5)	463 episodes	Robot force / torque after failure detection	[8] https://doi.org/10.24432/C5M89N (CC BY 4.0)
Universal Robots — Understanding Protective Stops	Manufacturer guidance	1 article	Payload / friction concepts	[1]
Universal Robots — C162 Error Code	Manufacturer error note	1 page	Incorrect payload / CoG example	[2]
Universal Robots — e-Series Service Handbook	Service manual (PDF)	Protective-stop flow	Troubleshooting reference	[3]
Project artefacts				
`dataset/raw/uci_robot_failures/*.data`	Original UCI files	5 files	Untouched source download	UCI archive
`dataset/raw/uci_robot_failures_instances.csv`	Parsed authentic table	463 instances	ETL extract output	Project file
`dataset/etl_cleaning.ipynb`	Jupyter ETL (pandas)	DataFrame pipeline	Cleaning + knowledge-map demo	Project file
`dataset/cleaned/*`	Cleaned information + report	451 sensor rows	Load for prediction	Project file
`knowledge_base.json`	Authored rule / frame KB	8 faults; 19 symptoms	Vocabulary, actions, provenance	Project file [5]
`sample_cases.json`	Synthetic test cases	13 scenarios	Regression + demonstration	Project file [5]
Supabase project tables	Cloud datastore	workspaces, cases, tickets	Optional shared persistence	[6]
Automated tests `test*.mjs`	Software checks	37 checks (3 suites)	Behaviour regression	Project file

6.A ETL and Cleaning (Dataset → Information)

The ETL step is a must-have for this project. The authentic source is the UCI Robot Execution Failures dataset — force and torque samples recorded after a failure is detected, with each failure described by 15 samples of six measurements (90 features) [8]. It is distributed as five learning problems, LP1–LP5 (Table 15). Local copies are kept under `dataset/raw/uci_robot_failures/`, with the citation in `dataset/raw/SOURCE.md`.

Table 15: *UCI Robot Execution Failures — the five learning problems*

Set	Failure type described	Instances	Share of 463
LP1	Failures in approach to grasp position	88	19.0 %
LP2	Failures in transfer of a part	47	10.2 %
LP3	Position of part after a transfer failure	47	10.2 %
LP4	Failures in approach to ungrasp position	117	25.3 %
LP5	Failures in motion with part	164	35.4 %
Total		463	100 %

The six ETL steps are summarised in Table 16. The pipeline is demonstrated in `dataset/etl_cleaning.ipynb` (pandas DataFrames) and visualised on the Dataset & ETL page of the application.

Table 16: *ETL pipeline steps*

Step	Name	What we do
1	Extract	Parse UCI LP1–LP5 `.data` files into an instance DataFrame (class + force / torque summaries).
2	Clean incomplete	Drop episodes with incomplete 15-sample windows.
3	Clean missing / outliers	Drop NaNs; remove extreme force / torque absolute-maximum outliers.
4	Deduplicate	Drop duplicate sensor fingerprints.
5	Knowledge map	Map UCI `failure_class` → educational KB symptoms (documented mapping table).
6	Load	Write cleaned CSVs and prediction cases → app Dataset & ETL page.

Measured cleaning result (current run): 463 UCI instances → 451 sensor rows kept (4 extreme outliers and 8 duplicates removed) → 785 mapped evidence rows → 5 LP-grouped prediction cases loaded into the UI.

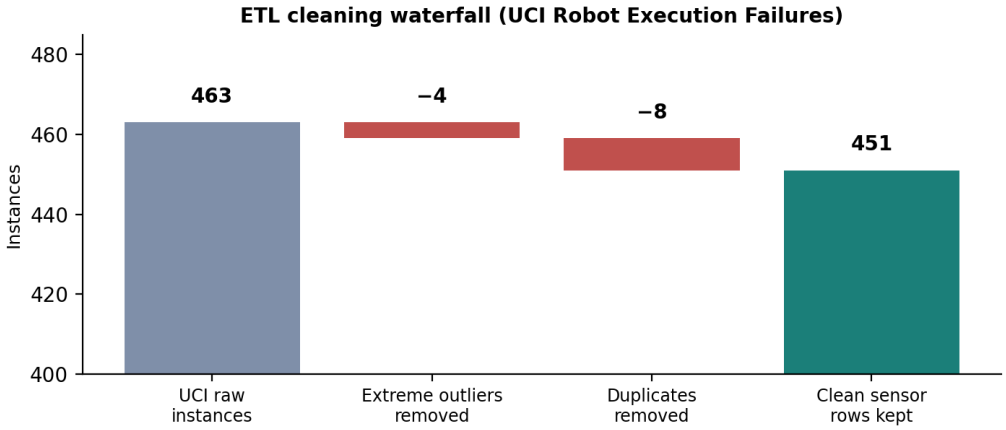


Figure 5: *ETL cleaning waterfall: 463 raw instances reduced to 451 clean sensor rows.*

Table 17: *Noise removed by the cleaning pipeline*

Drop reason	Count	Note
incomplete_ft_window	0	All episodes had full 15-sample windows
missing_class_or_features	0	UCI reports no missing values
extreme_sensor_outlier	4	Extreme force / torque absolute-maximum values
duplicate	8	Duplicate sensor fingerprints
unmapped_class	0	All failure classes map to a KB symptom

CLI refresh: `py -V:3.13 dataset/parse_uci_robot_failures.py` followed by `py -V:3.13 dataset/run_etl_notebook_logic.py`.

6. Implementation

6.1 Software / Tools / Libraries

Table 18: *Software, tools and libraries*

Tool	Role
HTML / CSS / JavaScript (ES modules)	Client-side KBS application
JSON / CSV	Knowledge base, sample cases, raw and cleaned dataset
Python 3 + pandas	`etl_cleaning.ipynb`, `run_etl_notebook_logic.py` and the `serve.py` local server
Node.js (optional)	CLI regression tests (`test*.mjs`)
Supabase JS (CDN ESM)	Optional cloud sync [6]
Vercel	Public static deployment of `dist/` [7]

6.2 Modules and Components

Table 19 lists the application modules; Figures 6–12 show the main screens (captured from the running application).

Table 19: *Application modules and their functions*

Module / screen	Function
Diagnostic studio	Enter observations; show fault predictions and the reasoning trace.
Dataset & ETL	Show cleaning steps, the noise-drop table and batch prediction.
Case history	Save, reopen and compare investigations (with a KB snapshot).
Maintenance board	Track Open / In progress / Completed tasks.
Rule explorer	Inspect and edit educational rules with validation.
Evaluation lab	Run the 13 authored scenarios.
Viva demo button	Scripted walkthrough: payload → conflict → evaluation.
Cloud sync	Push / pull workspace when Supabase keys are configured.

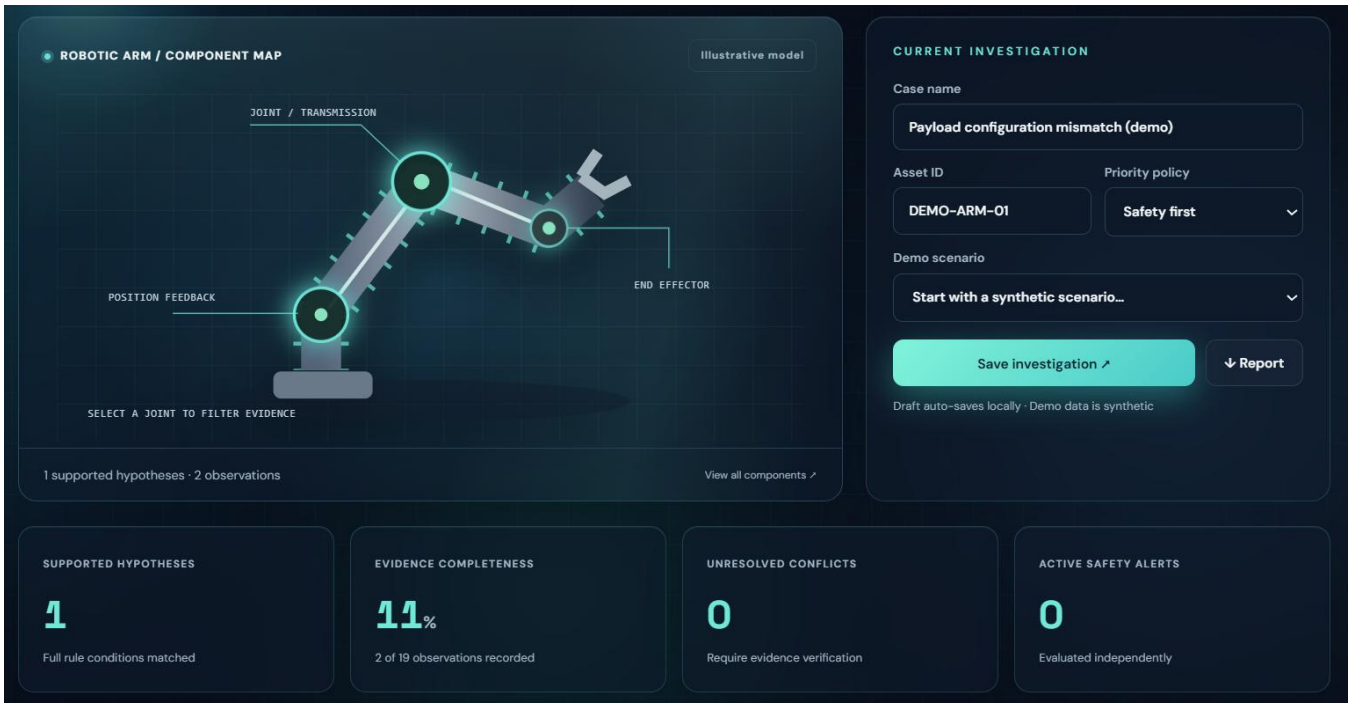


Figure 6: *Diagnostic studio — component map, current investigation and live metrics (payload scenario loaded).*

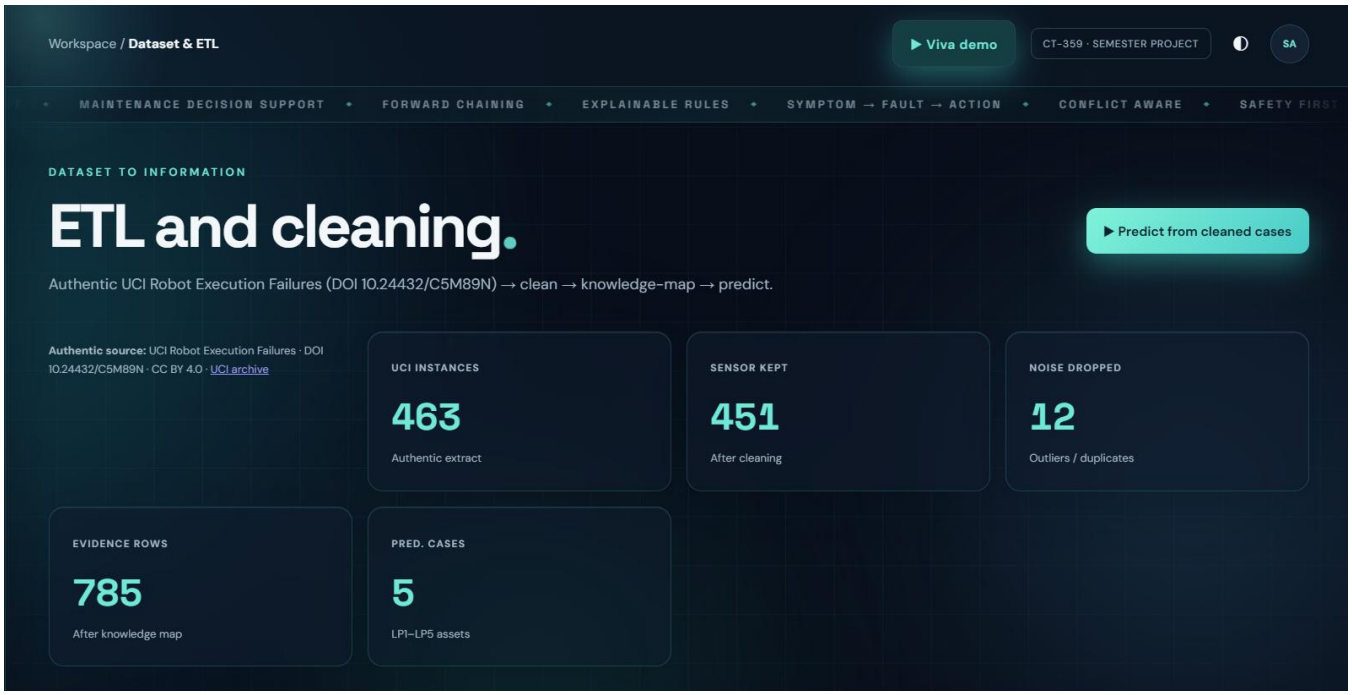


Figure 7: *Dataset & ETL page — authentic source citation and cleaning metrics (463 → 451 sensor rows, 785 evidence rows, 5 cases).*

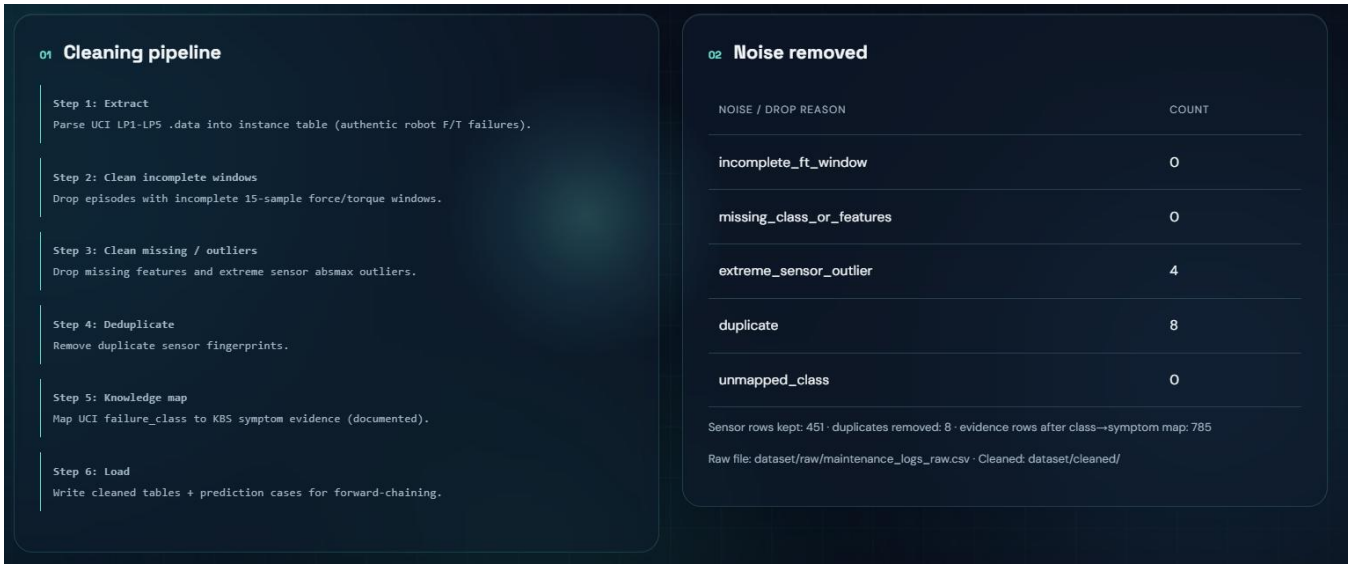


Figure 8: *Dataset & ETL page — six-step cleaning pipeline and the noise-removed table.*

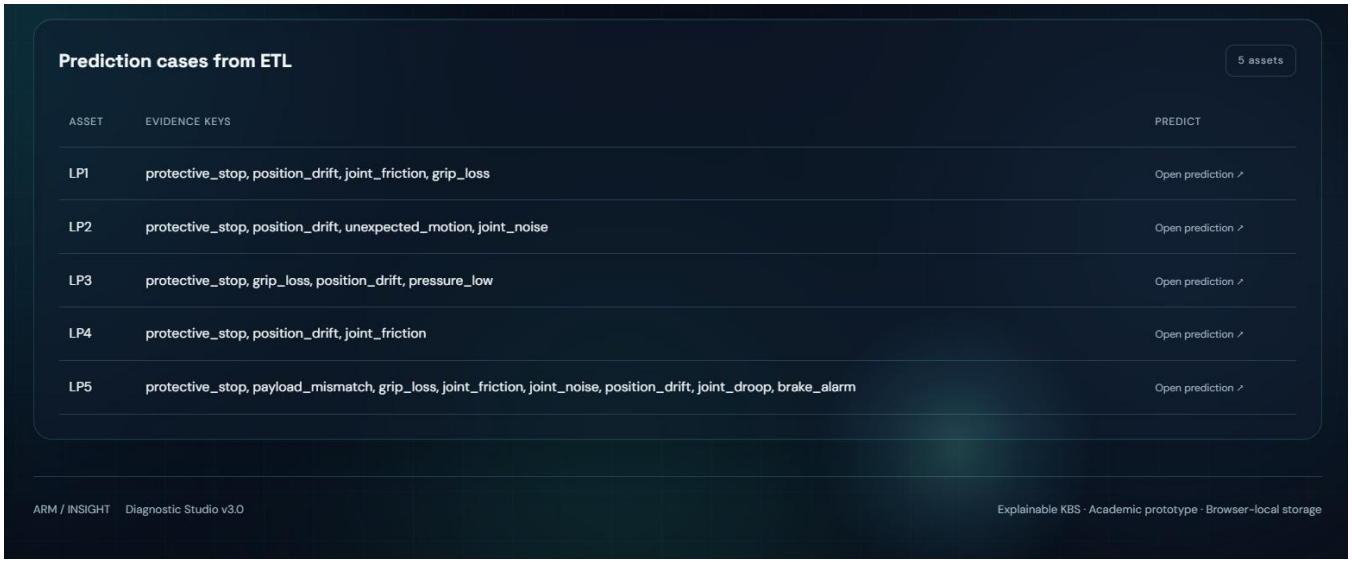


Figure 9: *Dataset & ETL page — batch predictions from the cleaned UCI asset cases (LP1–LP5).*

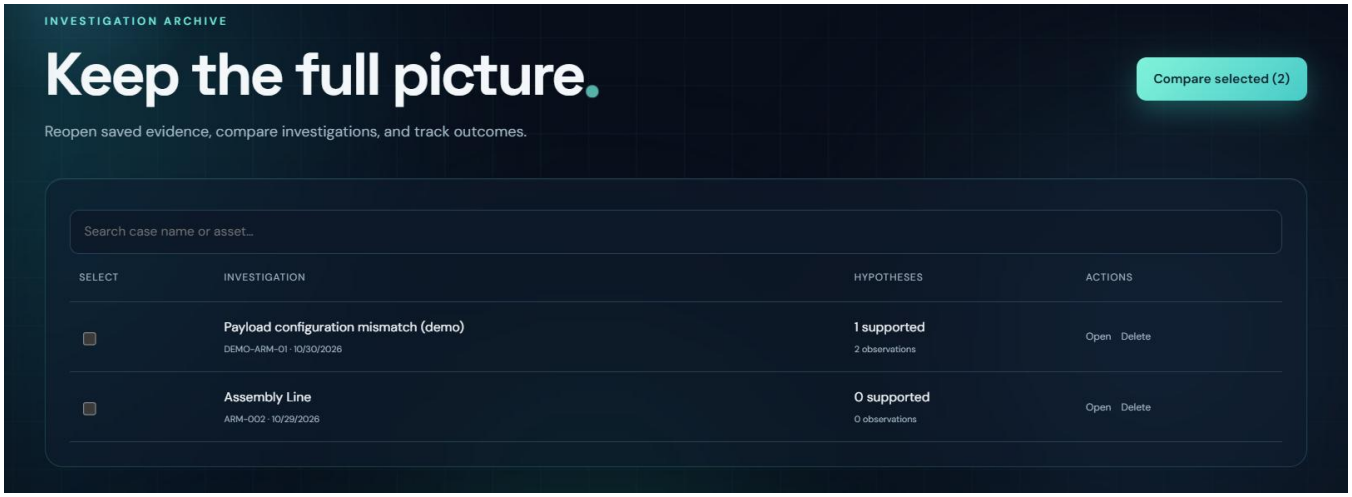


Figure 10: *Case history — saved investigation with search, selection and comparison.*

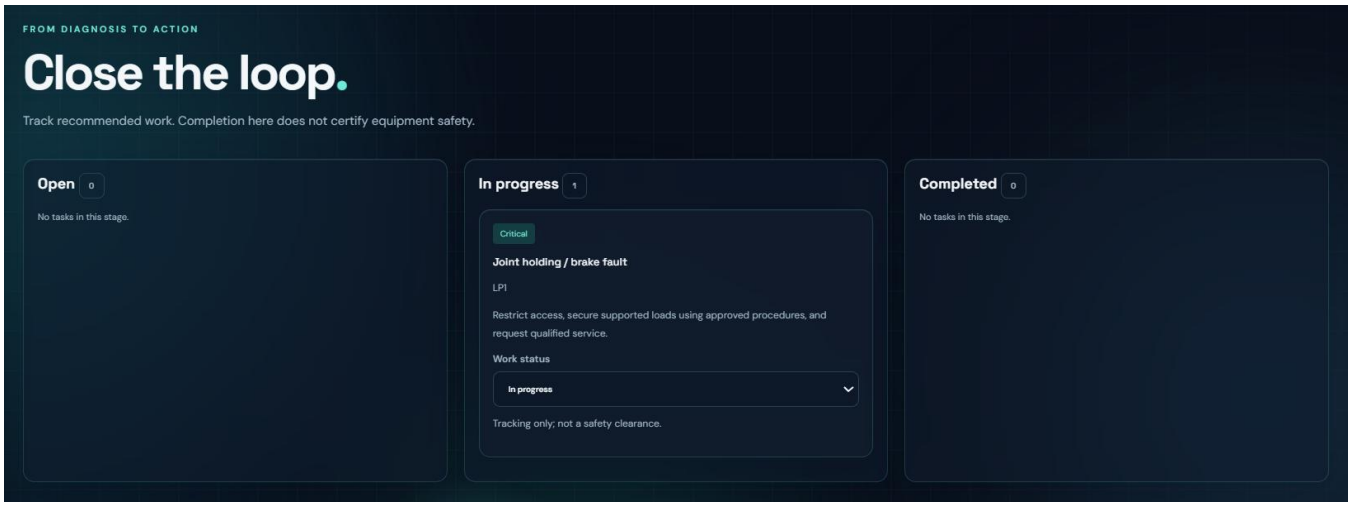


Figure 11: *Maintenance board — the recommended task appears in the Open column of the Open / In progress / Completed workflow.*

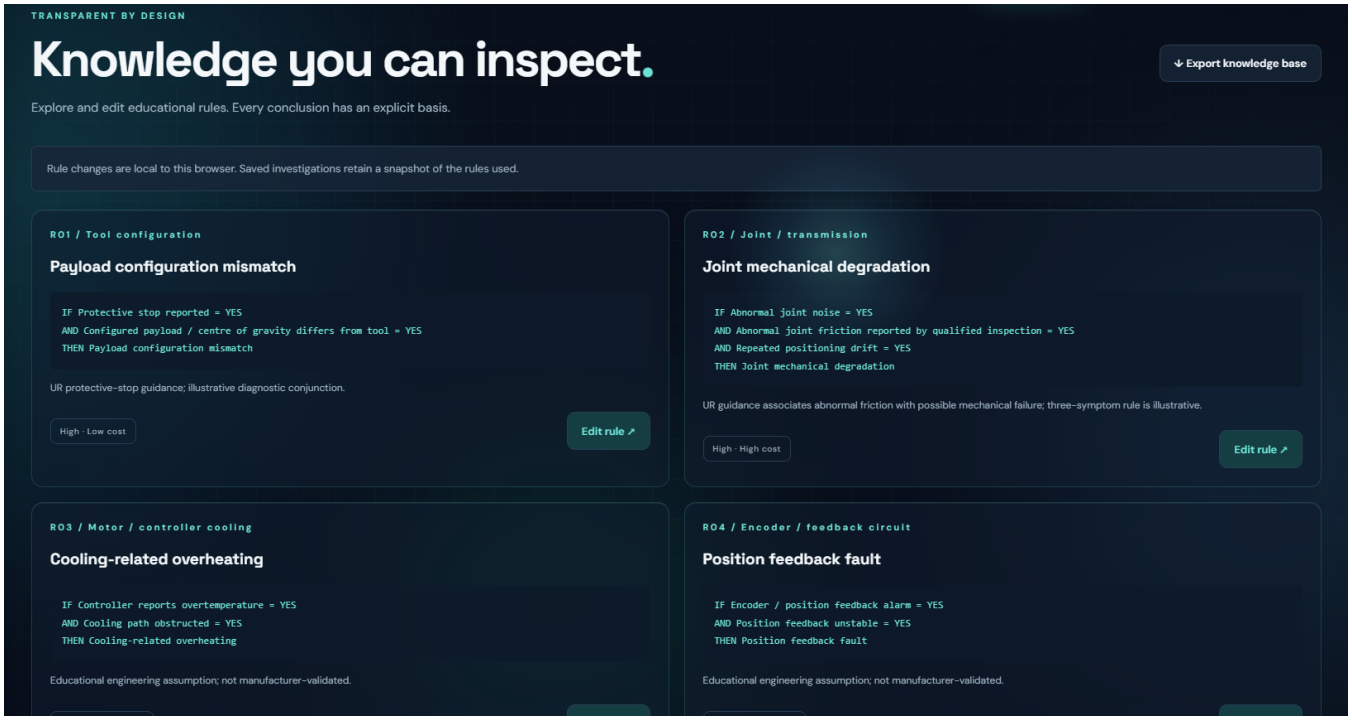


Figure 12: *Rule explorer — every diagnostic rule with its conditions, severity and provenance.*

6.3 Code Structure

Table 20: *Repository structure and file roles*

Path	Role
`dataset/raw/uci_robot_failures/`	Authentic UCI LP1–LP5 source
`dataset/cleaned/`	Cleaned data + ETL report
`dataset/etl_cleaning.ipynb`	Extract → clean → map → load

Path	Role
`dist/index.html`	Shell UI, ambient motion layer, splash
`dist/app.mjs`	Events, rendering, ETL UI, predictions
`dist/engine.mjs`	`diagnose()` — inference / prediction core
`dist/evidence.mjs`	Claim merge + validation
`dist/state.mjs`	Workspace / KB validation
`dist/cloud.mjs`	Supabase pull / push
`dist/config.mjs`	Supabase URL / anon key (empty = offline)
`dist/knowledge_base.json`	Frames + rules
`dist/etl_cleaning_report.json`	UI copy of the cleaning audit
`dist/prediction_cases.json`	UI copy of the ETL prediction cases
`dist/sample_cases.json`	Synthetic scenarios
`dist/style.css` / `polish.css`	Layout + glass / motion styling
`serve.py`	Localhost server
`supabase/schema.sql`	Tables + educational RLS policies
`vercel.json`	Publish `dist/`
`test.mjs`, `test-state.mjs`, `test-evidence.mjs`	Automated checks

6.4 User Inputs and Outputs

Table 21: *Inputs accepted and outputs produced*

Direction	Details
Inputs	Symptom states or multi-source claims; case name; asset ID; notes; priority policy (Safety first / Lower cost / Less downtime); optional rule edits; optional restore backup / cloud sync.
Outputs	Ranked hypotheses; next-best questions; safety alerts; reasoning trace; maintenance tasks; case comparison; text investigation report; KB export; workspace backup JSON; mirrored rows in Supabase when enabled.

7. Testing and Results

7.1 Synthetic Evaluation Lab (13 / 13)

All 13 authored scenarios matched the expected Supported fault-id sets under the default rules (Table 22, Figure 13).

Table 22: *Evaluation-lab scenarios and results*

#	Scenario	Expected supported	Result
1	Payload configuration mismatch	payload	PASS
2	Joint mechanical degradation	mechanical	PASS
3	Cooling-related overheating	thermal	PASS
4	Position feedback fault	encoder	PASS
5	Pneumatic grip fault	gripper	PASS
6	Communication connection fault	communication	PASS
7	Power supply fault	power	PASS
8	Joint holding / brake fault	brake	PASS
9	Insufficient evidence	(none)	PASS
10	Conflicting payload reports	(none)	PASS
11	Negative evidence	(none)	PASS
12	Two simultaneous faults	payload, gripper	PASS
13	Immediate safety escalation	(none) + safety alert	PASS

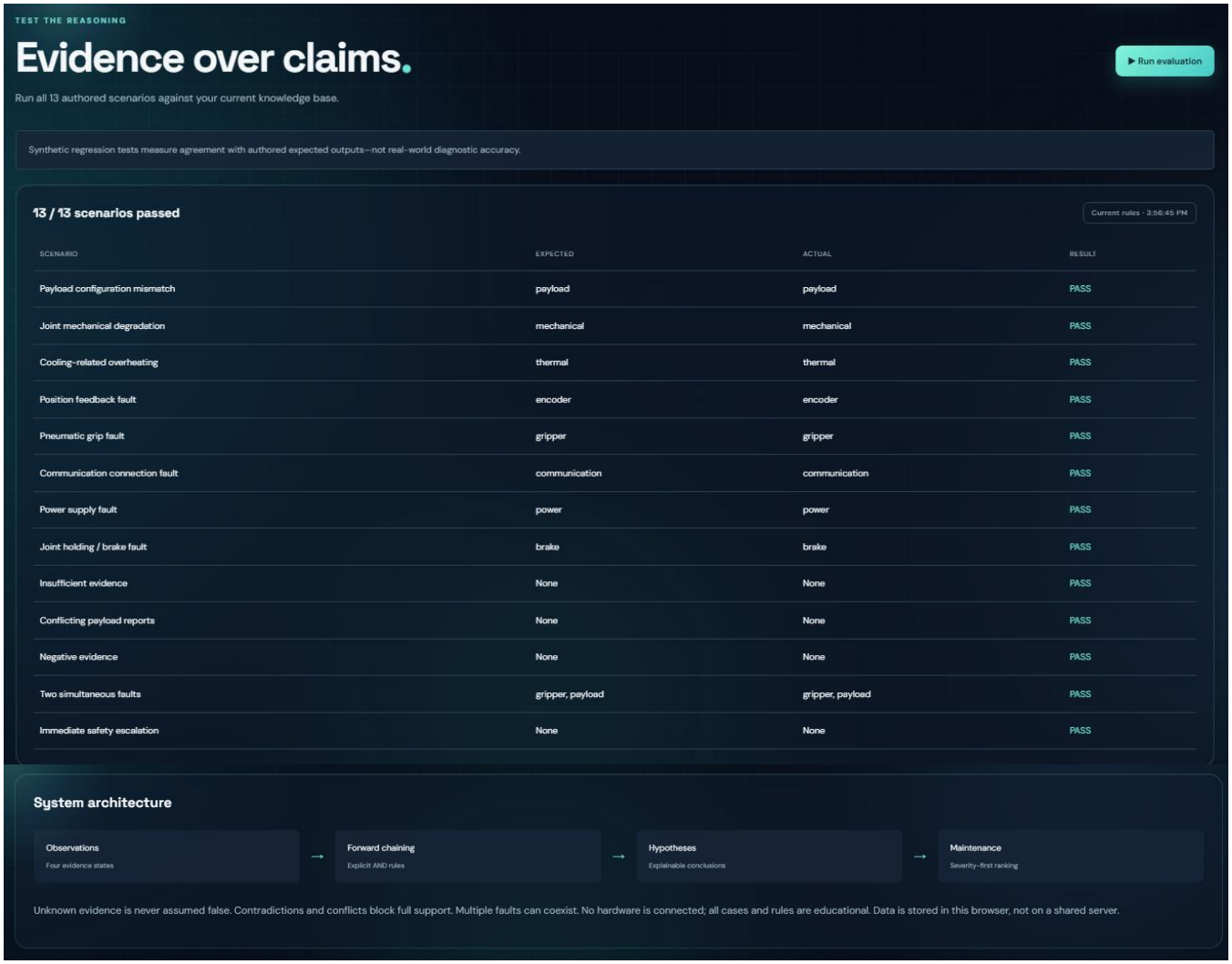


Figure 13: *Evaluation lab after "Run evaluation": all scenarios pass with expected and actual hypotheses shown.*

7.2 Automated Node Checks

The three CLI suites were re-run for this report with `node test.mjs`, `node test-state.mjs` and `node test-evidence.mjs`:

Table 23: *Automated test-suite output*

Suite	Output	Checks
`test.mjs`	13 sample scenarios and 8 inference checks passed.	21
`test-state.mjs`	9 workspace and rule validation checks passed.	9
`test-evidence.mjs`	7 evidence-source tests passed.	7
Total		37 — all passed

The inference checks verify Possible vs Supported, Contradicted, Conflicted, hazard alerts, multi-fault ordering under each policy and invalid-evidence rejection; the state and evidence suites verify workspace / rule validation and source-merge behaviour. These check software behaviour, not field diagnostic accuracy.

7.3 Manual and Demonstration Checks

Key behaviours were also verified in the running application (Figures 14–17).

- Demo scenario loads findings and trace.
- Operator Yes + Sensor No → Conflict blocks support.
- Case save survives refresh (localStorage); Sync mirrors to Supabase when configured.
- Guided mode advances unanswered checks.
- Case compare, maintenance-task status, and rule edit + re-run evaluation.
- Deployed URL loads the same static client.

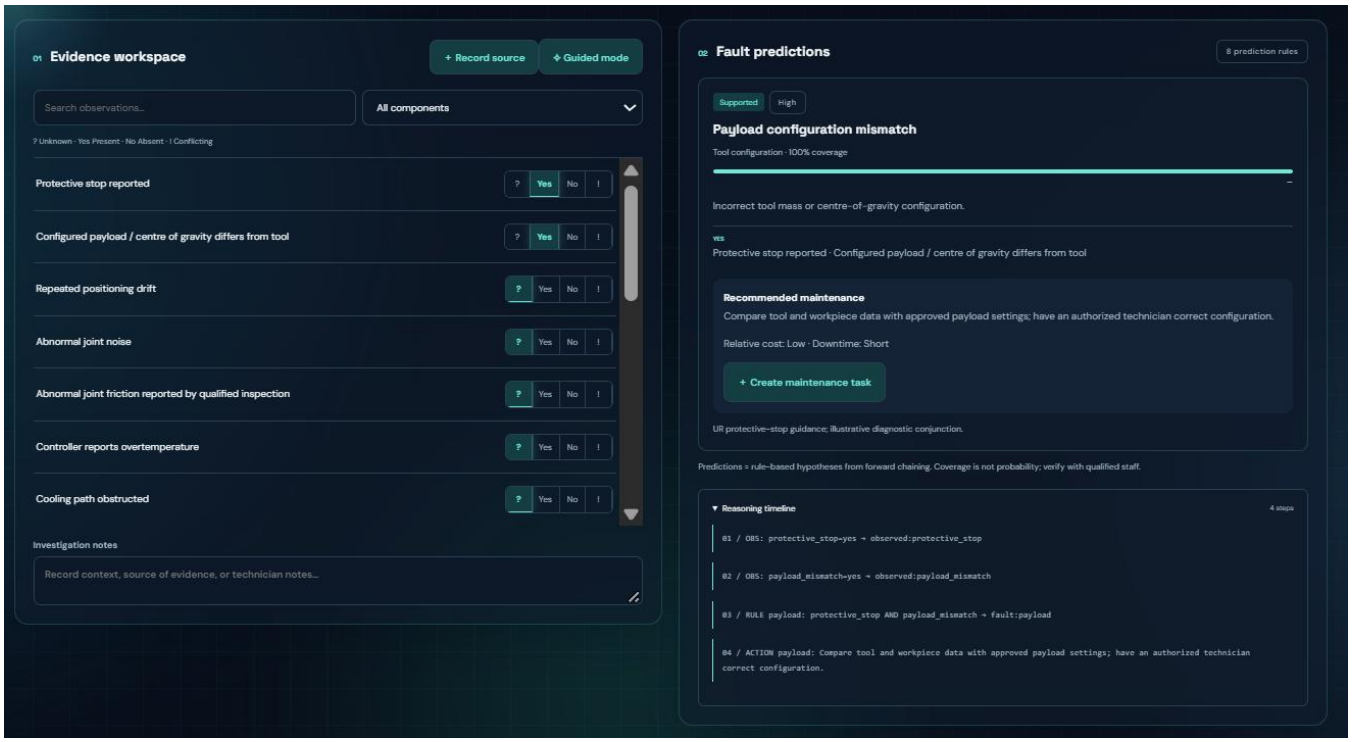


Figure 14: *Supported fault: protective_stop = Yes and payload_mismatch = Yes fire the payload rule, with the cause, recommended action and the 4-step reasoning timeline.*

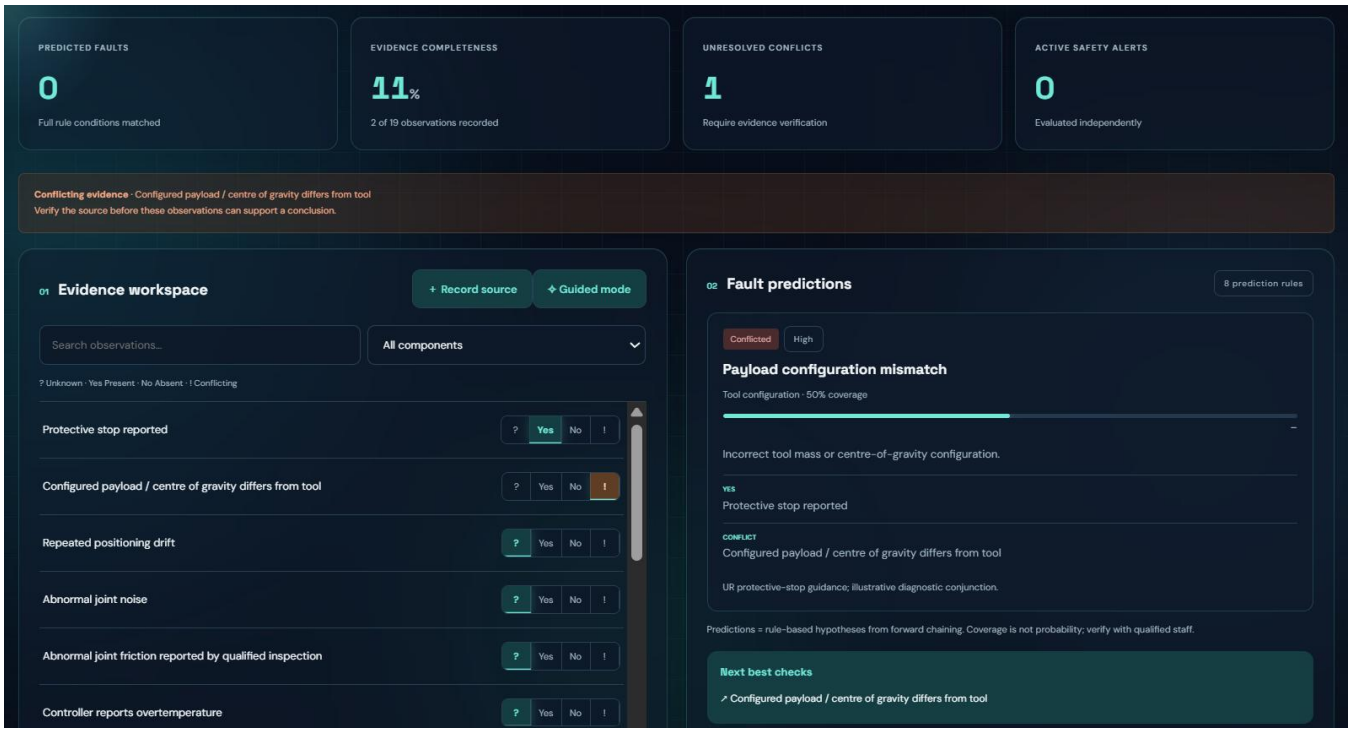


Figure 15: *Conflict handling: the payload symptom is Conflicting, so the hypothesis is "Conflicted" with 50 % coverage and a next-best check is suggested.*

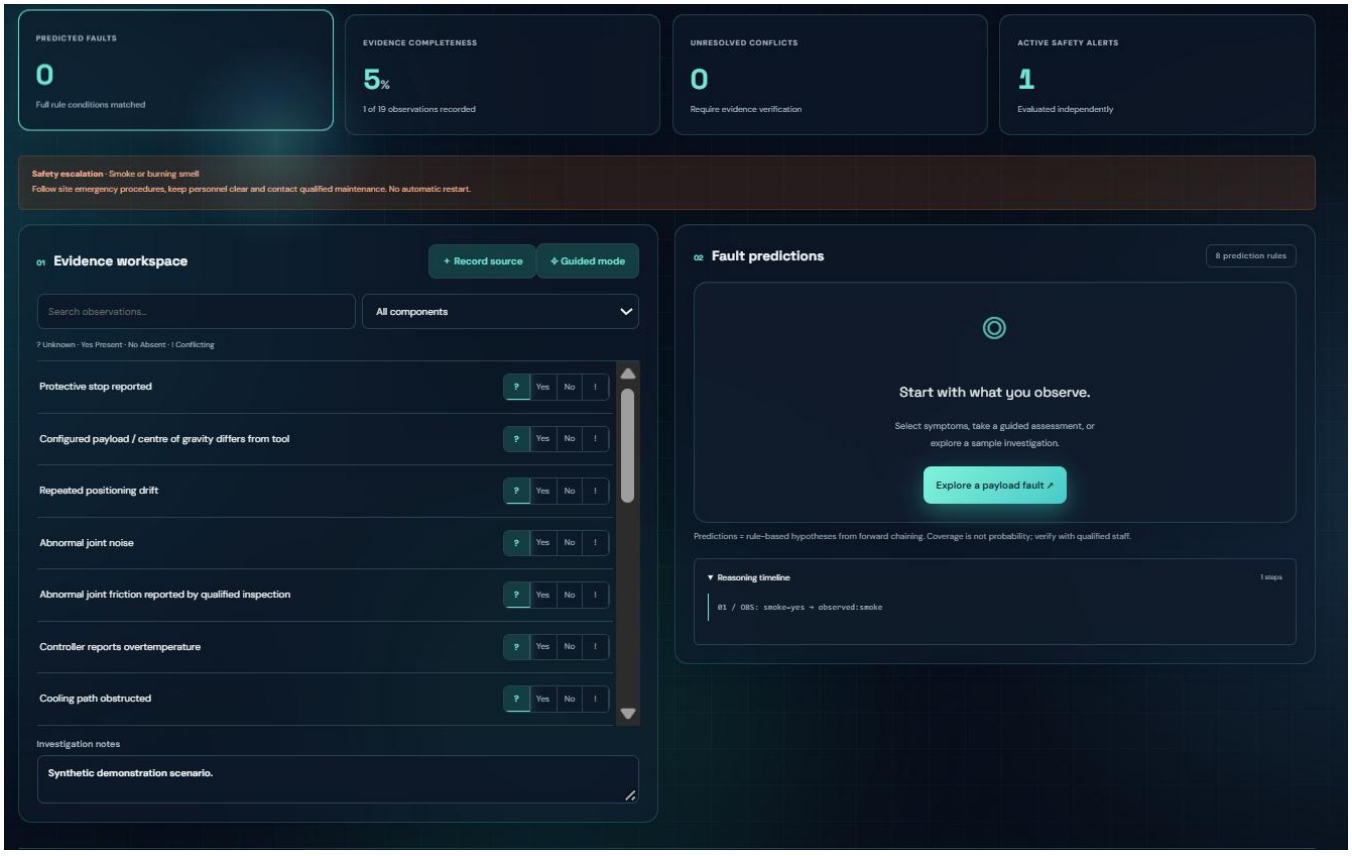


Figure 16: *Immediate safety escalation: hazard symptoms raise an independent alert even when no fault rule is Supported.*

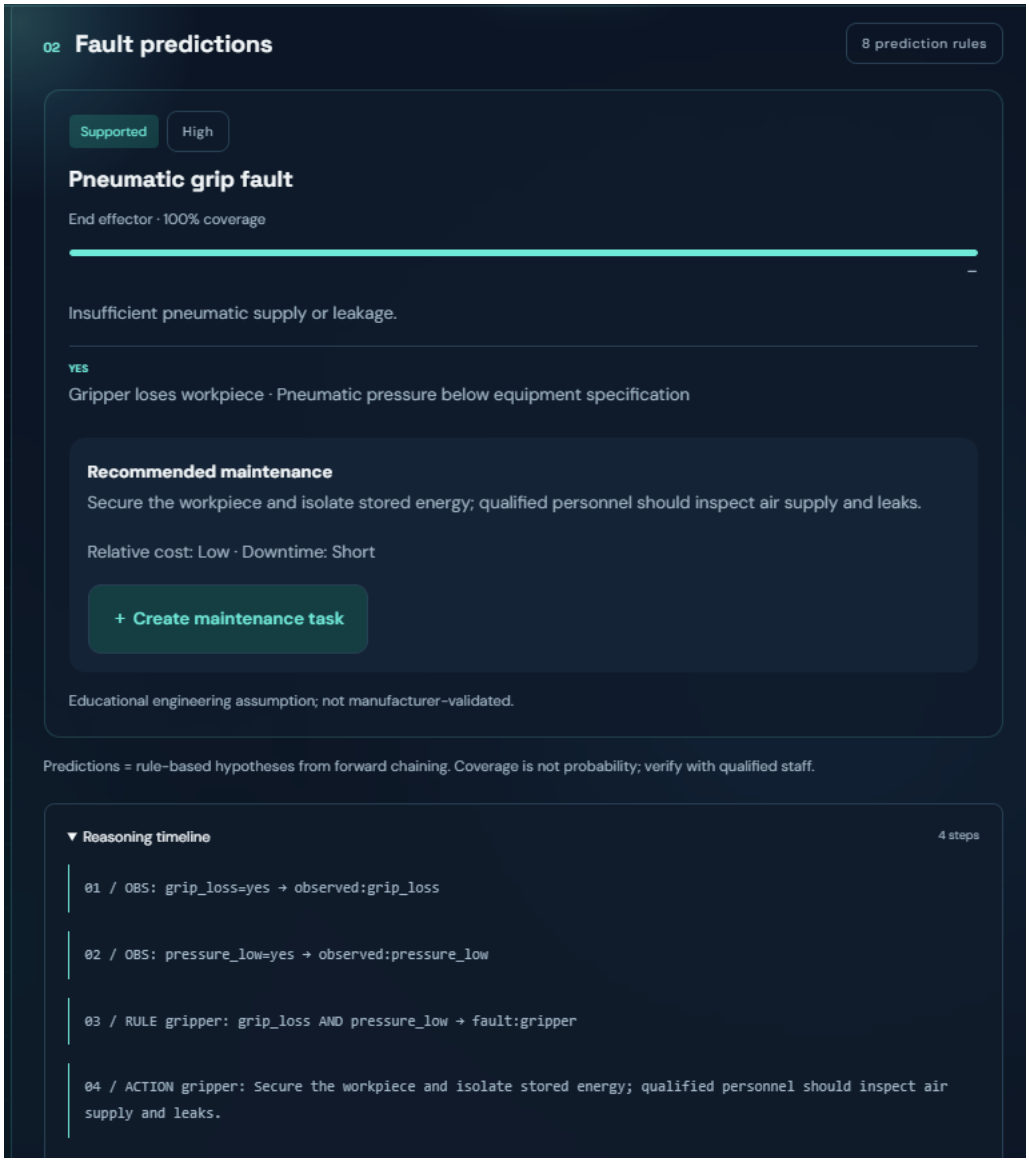


Figure 17: *Two simultaneous faults: payload and gripper rules are both Supported and listed together with their actions.*

8. Evaluation, Limitations and Conclusion

8.1 Evaluation of the Proposed Approach

Table 24 compares the chosen rule-based forward-chaining approach with the main alternatives against the criteria named in CLO 3.

Table 24: *Evaluation of rule-based forward chaining against alternatives (CLO 3)*

Criterion	Rule-based forward chaining (chosen)	Alternative / trade-off
Explainability	Every condition and derived action is inspectable.	ML needs labels and post-hoc explanation.
Incomplete evidence	Unknown ≠ No; rules stay Possible.	Bayesian methods need calibrated probabilities (not supplied).
Safety	Independent hazard alert and severity-first ranking.	Still requires human procedures.
Cost / availability	Secondary qualitative ordering.	Needs real plant numbers.
Maintainability	Editable JSON rules and a regression suite.	A large ontology / graph may scale better across many asset types.
Screening style	Natural for "given these observations, what fires?"	Backward chaining is better if a single target fault is already suspected.

8.2 Limitations

- Generic arm; only eight fault hypotheses.
- Qualitative cost / downtime; no live telemetry or model-specific thresholds.
- Source categories are not authenticated or credibility-weighted.
- Conflicts wait for human resolution.
- Supabase demo policies are intentionally open (educational) — not production security.
- Regression PASS ≠ industrial accuracy or safety clearance.

8.3 Future Improvements

1. Bind knowledge to a **specific** robot model and approved manuals.
2. Acquire expert-labelled incident cases; measure precision / recall and missed-hazard rate.
3. Import timestamped alarms; weight source reliability.
4. Add authentication and audit logging on the shared store.
5. Optionally enrich with a lightweight ontology / graph view for cross-equipment reuse, while keeping rule explanation.

8.4 Conclusion

ARM / INSIGHT demonstrates how a KBS can represent industrial robotic-arm knowledge, analyse incomplete stakeholder requirements and apply justified forward-chaining inference with conflict handling and explanations. Synthetic tests and a deployable prototype support course demonstration. The system's academic value is transparency; real deployment requires model-specific knowledge acquisition and independent validation.

9. References

[1] Universal Robots, “Understanding Protective Stops.” https://www.universal-robots.com/articles/ur/robot-care-maintenance/understanding-protective-stops/ (accessed 2026-10-23).

[2] Universal Robots, “C162 Error Code.” https://www.universal-robots.com/manuals/EN/HTML/SW5_19/Content/prod-err-codes/topics/CODE_162.html (accessed 2026-10-23).

[3] Universal Robots, “e-Series Service Handbook — Protective Stop.” https://www.universal-robots.com/manuals/EN/PDF/SW5_24/service-manual_e-series/e-Series_Service_Manual_en.pdf (accessed 2026-10-23).

[4] NED University of Engineering and Technology, Knowledge-Based Systems (CT-359) Complex Computing Problem brief and report template, 2026.

[5] ARM / INSIGHT project package: knowledge_base.json, sample_cases.json, inference engine and tests, version 2.1, 2026. https://github.com/MTahaFarrukh/arm-insight-kbs

[6] Supabase documentation — client libraries and Row Level Security (project educational use), 2026.

[7] Vercel documentation — static site deployment, 2026.

[8] L. Lopes and L. Camarinha-Matos, “Robot Execution Failures” [Dataset], UCI Machine Learning Repository, 1998. https://doi.org/10.24432/C5M89N (CC BY 4.0).

Appendix A. Requirement Traceability

Table 25: *Requirement traceability matrix*

Brief requirement	Implementation evidence
Knowledge representation (CLO 1)	JSON symptom dictionary; eight component / fault / cause / action frames.
Problem analysis (CLO 2)	Section 4 — stakeholders, incomplete requirements, constraints, assumptions.
Algorithm assessment (CLO 3)	Sections 5.3 and 8.1 — forward chaining vs alternatives.
Conflicting information	Multi-source Yes + No → Conflict; blocks Supported.
Explanation and safety	Trace, condition groups, independent hazard alerts.
Dataset and results	UCI 463 episodes + ETL; 13 / 13 evaluation PASS; 37 CLI checks PASS.
Deployable demo	https://arm-insight-kbs-dist.vercel.app/ and localhost `serve.py`.